

Лабораторная работа №12

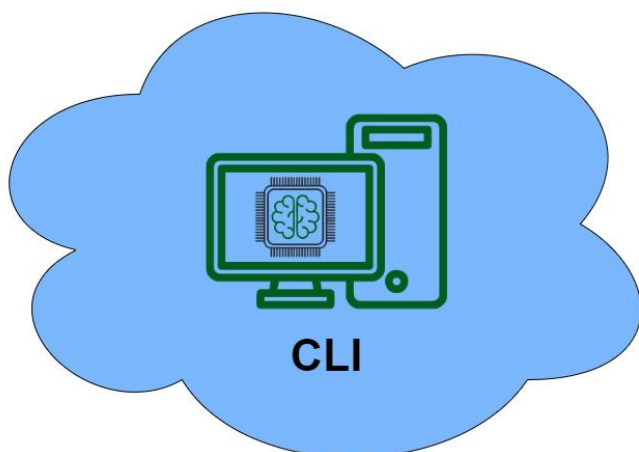
Разграничение доступа

Цель работы:

Изучить и освоить основные механизмы разграничения доступа в операционной системе Linux, включая работу с правами доступа к файлам и каталогам

Оборудование, ПО:

- CLI
- Справочная литература или доступ в сеть интернет.



Машина	IP	Маска	Шлюз	DNS
CLI (Альт Рабочая станция 10.4)	192.168.1.1	24	-	77.88.8.8

Порядок работы:

1. Переименовать ПК своей фамилией
2. Вставить скриншоты результатов работы каждой команды

Контрольные вопросы:

1. Какие типы прав доступа существуют в Linux для файлов и каталогов?
2. Что означают символы в строке прав (например, drwxr-xr--)?
3. Как изменить права доступа с помощью команды `chmod`?
4. Как создаются и настраиваются группы пользователей в Linux?
5. Чем отличается команда `chown` от `chmod`?
6. Что такое ACL и зачем он нужен?
7. Как работает механизм `umask`?
8. Что такое SUID, SGID и Sticky bit?
9. Какие утилиты используются для управления доступом на уровне процессов или служб?

Используемые источники:

- Официальная документация операционной системы Альт Рабочая Станция 10.4

<https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html/alt-workstation/index.html>

Задание:

1. Создайте пользователей с соответствующими паролями и должностями:

Логин	Пароль	Должность
max	vifluk7OcwybOkOd	разработчик
peter	noinheesEnCystac	системный администратор
vadim	ElOddupodyeedcaj	аудитор
boris	BujcewlyadHomHas	потенциальный нарушитель

2. Создайте группы и включите в них соответствующих пользователей:

Группа	Пользователи, входящие в группу
developers	max
sysadmins	peter
auditors	vadim
restricted	boris

Проверьте, что пользователи успешно были добавлены в свои группы.

3. Создайте каталоги и настройте права доступа к ним:

Каталог `/code_repo` должен быть доступен только для группы `developers` с полным доступом, при этом остальные пользователи не должны иметь к нему доступ. Настройте каталог `/code_repo` таким образом, чтобы все новые файлы наследовали группу владельца.

Каталог `/admin_tools` должен быть доступен исключительно для группы `sysadmins` с полным доступом, при этом остальные пользователи не должны иметь к нему доступ.

Каталог `/audit_logs` должен быть доступен только для группы `auditors` с полным доступом, при этом остальные пользователи не должны иметь к нему доступ.

Настройте каталог `/shared_files` таким образом, чтобы пользователи могли удалять только свои файлы.

Проверьте корректность настроек, попытавшись получить доступ к каждому каталогу с учетной записи пользователя, не входящего в соответствующую группу.

1. Теоретическая основа управления доступом

Модель субъектно-объектного взаимодействия является основой для понимания управления доступом в ОС. В этой модели различают две основные сущности: субъект и объект.

Субъект — активная сущность, которая инициирует действия по доступу к объектам.

Субъектами могут быть пользователи, процессы, а также системные службы.

Объект — пассивная сущность, к которой осуществляется доступ.

Объектами могут быть файлы, каталоги и устройства.

Матрица доступа — таблица, которая формализует права доступа субъектов к объектам.

В качестве примера приведем матрицу управления доступом, в которой определены права субъектов на объекты.

Субъектами являются пользователи *alex*, *sophia* и группа *support*.

В качестве объектов выделены файл логов *server_logs.txt*, скрипт резервного копирования *backup_script.py*, каталог *admin_tools*.

Таблица 1 — Пример матрицы управления доступом

Субъект/Объект	server_logs.txt	backup_script.py	admin_tools
alex	rw-	r--	rwX
sophia	---	rwX	r-X
support	r--	---	rw-

2. Дискреционное управление доступом (DAC)

Discretionary Access Control (DAC) — модель управления доступом, основанная на полномочиях владельца объекта. Владелец определяет, какие субъекты могут взаимодействовать с объектом, а также устанавливает для них конкретные разрешения.

В мандатных системах доступа (MAC) владельцы не могут свободно управлять своими объектами. Доступ контролируется метками безопасности и глобальными политиками.

В системе DAC выделяют три ключевых субъекта управления доступом:

1. Владелец (User, u) — пользователь, создавший объект, автоматически становится его владельцем.
2. Группа (Group, g) — пользователи, объединенные в одну группу.
3. Прочие (Others, o) — все остальные пользователи системы.

Таблица 2 — Базовые права доступа

Разрешение	Влияние на файлы	Влияние на каталоги
r (чтение)	Разрешено читать содержимое файлов	Разрешено отображать содержимое каталога (имена файлов)

w (запись)	Разрешено изменять содержимое файлов	Разрешено создавать и удалять любые файлы в каталоге
x (выполнение)	Разрешено запускать файлы как программы	Разрешено переходить в каталог

3. Управление базовыми разрешениями

Чтобы получить информацию о правах доступа, используйте следующую команду:

```
$ ls -l
```

При этом будет выведена подробная информация о файлах и каталогах, в которой будут, среди прочего, отражены права доступа.

Зайдем под учетной записью суперпользователя, после чего создадим учетную запись *testuser* с паролем *P@ssw0rd*.

Затем зайдем под учетной записью *testuser* и создадим в домашнем каталоге файл *file.txt* с текстом “Проверка прав доступа”, также создадим каталог *demodir*.

```
# mkdir demodir
```

Теперь выведем информацию о правах доступа:

```
$ ls -l
```

```
[testuser@alt ~]$ ls -l
итого 4
drwxr-xr-x 2 testuser testuser 4096 demodir
-rw-r--r-- 1 testuser testuser 0 file.txt
```

Первый символ указывает на тип файла. Это может быть обычный файл, каталог (d), символическая ссылка (l) или файлы других специальных типов. Следующие девять символов представляют права доступа к файлам, три тройки по три символа в каждой. Первая тройка показывает разрешения владельца, вторая — групповые разрешения, а последняя тройка показывает разрешения всех остальных. Если буква заменена на -, то значит разрешение не предоставляется. Первое имя указывает на владельца файла, а второе — на владельца-группу.

Выполним команду *ls* с опцией *-ld*, чтобы отобразить подробную информацию о каталоге, но не о его содержимом:

```
$ ls -ld
```

```
[testuser@alt ~]$ ls -ld
drwx----- 9 testuser testuser 4096 .
```

Для изменения разрешений из командной строки используется команда *chmod*, которая означает “*change mode*” (смена режима).

Команда *chmod* принимает инструкцию разрешения, за которой следует список файлов или каталогов, которые необходимо изменить. Инструкция разрешения может быть отправлена в символьном или числовом виде.

Синтаксис команды *chmod* (символьный метод):

```
$ chmod [options] WhoWhatWhich file | directory
```

где *Who*: *u* — *user*, *g* — *group*, *o* — *other*, *a* — *all*;

What: + добавление, - удаление, = точная установка;

Which: *r* — чтение, *w* — запись, *x* — выполнение.

Для символического метода не обязательно задавать полный набор разрешений.

Команда *chmod* поддерживает опцию *-R* для рекурсивной установки разрешений файлов во всем дереве каталогов.

Зайдем в учетную запись *testuser*.

Удалим для группы *testuser* и других пользователей разрешения на чтение и выполнение в каталоге *demodir*.

```
$ chmod go-rx demodir
```

Добавим для всех разрешение на выполнение файла *file.txt*:

```
$ chmod a+x file.txt
```

ИЛИ

```
$ chmod +x file.txt
```

Проверяем изменения:

```
$ ls -l file.txt
```

```
[testuser@alt ~]$ ls -l file.txt
-rwxr-xr-x 1 testuser testuser 0 file.txt
```

Синтаксис команды *chmod* (числовой метод):

```
$ chmod [options] ??? file | directory
```

В числовом методе разрешения представлены восьмеричным числом из 3-х разрядов (или 4-х в случае настройки дополнительных разрешений). Один такой разряд может представлять любое значение от 0 до 7. Также в трехразрядном восьмеричном (числовом) каждый разряд обозначает один уровень доступа (слева направо): пользователь, группа и остальные.

Разряд ? рассчитывается путем сложения цифр каждого разрешения, которое необходимо добавить: 4 — чтение, 2 — запись, 1 — выполнение.

В качестве примера разберем разрешение 744:

Владелец: $rwx = 4+2+1 = 7$

Группа: $r-- = 4+0+0 = 4$

Прочие: $r-- = 4+0+0 = 4$

В результате получается трехзначное значение 744.

Установим для пользователя *testuser* разрешение на чтение и запись файла *file.txt*, для группы *testuser* — только на чтение, а для остальных пользователей не предоставим разрешений:

```
$ chmod 640 file.txt
```

Проверяем изменения:

```
$ ls -l file.txt
```

```
[testuser@alt ~]$ ls -ld  
drwx----- 9 testuser testuser 4096
```

Каждые новые файлы принадлежат создавшему его пользователю. Также по умолчанию новые файлы принадлежат к основной группе пользователя, который создал файл. В ОС Альт основная группа пользователя обычно является частной группой, которая состоит только из одного этого пользователя. Чтобы предоставить доступ к файлу на основе участия в группе, может потребоваться изменить группу, являющуюся владельцем файла.

Только пользователь *root* может изменить владельца файла! Однако назначить группу-владельца может как пользователь *root*, так и владелец файла. Пользователь *root* может назначить группу владельцем файла, только если они входят в эту группу.

Изменить владельца файла можно с помощью команды *chown*, которая означает “change owner” (смена владельца).

Зайдем под учетной записью суперпользователя и изменим владельца файла *file.txt* на *user*. Воспользуемся опцией *-c* для подробного вывода всех выполняемых изменений:

```
# chown -c user file.txt
```

```
[root@alt ~]# cd /home/testuser/  
[root@alt testuser]# chown -c user file.txt  
изменён владелец 'file.txt' с testuser на user
```

Проверим изменения:

```
# ls -l file.txt
```

```
[root@alt testuser]# ls -l file.txt  
-rw-r----- 1 user testuser 0 file.txt
```

Теперь изменим группу-владельца на root для файла *file.txt*:

```
# chown -c .root file.txt
```

```
[root@alt testuser]# chown -c .root file.txt  
изменён владелец 'file.txt' с user:testuser на :root
```

Проверим изменения:

```
# ls -l file.txt
```

```
[root@alt testuser]# ls -l file.txt  
-rw-r----- 1 user root 0 file.txt
```

Также для изменения группы-владельца можно использовать команду *chgrp*.

Синтаксис команды *chgrp*:

```
$ chgrp [options] group file | directory
```

Для выполнения команды пользователю необходимо иметь соответствующие права на изменение группы файла.

Вернем владельца и владельца-группу одной командой для файла *file.txt*:

```
# chown -c testuser:testuser file.txt
```

```
[root@alt testuser]# chown -c testuser:testuser file.txt  
изменён владелец 'file.txt' с user:root на testuser:testuser
```

Проверим изменения:

```
# ls -l file.txt
```

```
[root@alt testuser]# ls -l file.txt  
-rw-r----- 1 testuser testuser 0 file.txt
```

Команду *chown* можно использовать с опцией *-R* для рекурсивного изменения всего дерева каталогов.

4. Специальные разрешения

Если необходимо помимо основных типов разрешений задать дополнительные функции, то можно воспользоваться специальными разрешениями: *SUID*, *SGID*, *Sticky Bit*. Расширенные права доступа позволяют гибко управлять запуском программ и модификацией файлов пользователями.

Важно! Используйте специальные права доступа с осторожностью и только в ситуациях, где это хорошо обосновано и не создает уязвимостей.

Таблица 3 — Расширенные права доступа

Специальное разрешение	Влияние на файлы	Влияние на каталоги
SUID	Файл выполняется от имени пользователя, которые владеет файлом, а не пользователя, который вызвал его выполнение	Не влияет
SGID	Файл выполняет от имени группы, которая владеет файлом	Для новых файлов, созданных в каталоге, в качестве группы-владельца задается группа-владелец каталога
Sticky Bit	Не влияет	Пользователи с разрешением на запись в каталог могут удалять только те файлы, которыми они владеют. Они не могут удалять или принудительно сохранять данные в файлы, принадлежащие другим пользователям.

Расширенные права доступа также могут быть заданы в символьном или числовом виде.

В числовом методе специальный бит добавляет четвертой цифрой в восьмеричном представлении прав доступа.

Таблица 4 — Расширенные права доступа (числовой метод)

Число	Символ	Специальное разрешение
4	u+s	SUID
2	g+s	SGID
1	o+t	Sticky Bit

Разрешение *SUID* позволяет пользователю запускать исполняемый файл с правами владельца этого файла. Другими словами, использование этого бита позволяет нам поднять привилегии пользователя в случае, если это необходимо.

Выведем список файлов *SUID*:

```
# find / -perm -u=s -type f 2>/dev/null
```



```
[root@alt ~]# find / -perm -u=s -type f 2>/dev/null
/bin/umount
/bin/mount
/bin/su
/usr/sbin/hddtemp
/usr/bin/sudoreplay
/usr/bin/cdrdao
/usr/bin/libgtop_server2
/usr/bin/wodim
/usr/bin/fusermount3
/usr/bin/mtr
/usr/bin/pkexec
/usr/bin/bwrap
/usr/bin/sudo
/usr/bin/Xorg
/usr/bin/readom
/usr/lib/consolehelper/priv/auth
/usr/libexec/sss/krb5_child
/usr/libexec/sss/ldap_child
/usr/libexec/polkit-1/polkit-agent-helper-1
/lib/dbus-1/dbus-daemon-launch-helper
```

Классический пример использования *SUID*-разрешения — команда `mount`:

```
# ls -l /bin/mount
```

```
[root@alt ~]# ls -l /bin/mount
-rws--x--x 1 root root 63672 /bin/mount
```

На месте, где обычно установлен классический бит `x` (на исполнение), выставлен специальный бит `s`.

Добавим разрешение *SUID* для файла `file.txt`:

```
# chmod u+s file.txt
```

Проверяем изменения:

```
# ls -l file.txt
```

```
[root@alt testuser]# ls -l file.txt
-rwS----- 1 testuser testuser 0 file.txt
```

Обратим внимание, что вместо ожидаемой буквы `s`, видим заглавную `S`. Это случается, если *SUID* установлен, но сам владелец файла не имеет прав на его выполнение. Добавим это разрешение:

```
# chmod u+x file.txt
```

Проверяем изменения:

```
# ls -l file.txt
```

```
[root@alt testuser]# ls -l file.txt
-rws----- 1 testuser testuser 0 file.txt
```

Принцип работы *SGID* очень похож на *SUID* с отличием, что файл будет запускаться пользователем от имени группы, которая владеет файлом.

Выведем список файлов *SGID*:

```
# find / -perm -g=s -type f 2>/dev/null
```

Параметр `-perm` позволяет искать файловые объекты по установленным разрешениям.

Параметр `-type` позволяет искать файловые объекты определенного типа.

```
[root@alt ~]# find / -perm -g=s -type f 2>/dev/null
/usr/lib64/vte/gnome-pty-helper
/usr/bin/wall
/usr/bin/screen
/usr/bin/crontab
/usr/bin/ssh-agent
/usr/bin/gpg-agent
/usr/bin/gpg2
/usr/bin/passwd
/usr/bin/write
/usr/bin/at
/usr/bin/gpg
/usr/lib/chkpwd/tcb_chkpwd
/usr/lib/screen/tcb_chkpwd
/usr/lib/screen/utempter
/usr/lib/utempter/utempter
/usr/lib/consolehelper/helper
/usr/libexec/ping/ping
/usr/libexec/mate-screensaver-dialog
```

Иллюстрирует работу бита *SGID* команда *crontab*:

```
# ls -l /usr/bin/crontab
```

```
[root@alt ~]# ls -l /usr/bin/crontab
-rwx--s--x 1 root crontab 51728 /usr/bin/crontab
```

Добавим разрешение *SGID* для файла *file.txt*:

```
# chmod g+s file.txt
```

Не забываем выдавать права на исполнение группе-владельцу файла:

```
# chmod g+x file.txt
```

Проверяем изменения:

```
# ls -l file.txt
```

```
[root@alt testuser]# ls -l file.txt
-rws--s--- 1 testuser testuser 0 file.txt
```

Последнее специальное разрешение — *Sticky Bit*. В случае, если этот бит установлен для папки, то файлы в этой папке могут быть удалены только их владельцем. Примером использования этого разрешения — системная папка */tmp*. Папка */tmp* разрешена на запись любому пользователю, но удалять файлы в ней могут только пользователи, являющиеся владельцами этих файлов.

```
# ls -ld /tmp
```

```
[root@alt testuser]# ls -ld /tmp
drwxrwxrwt 15 root root 340 /tmp
```

Символ *t* указывает, что на папку установлено разрешение *Sticky Bit*.

Добавим разрешение *Sticky Bit* для каталога *demodir*:

```
# chmod o+t demodir
```

Не забываем выдавать права на исполнение группе-владельцу файла:

```
# chmod o+x demodir
```

Проверяем изменения:

```
# ls -ld demodir
```

```
[root@alt testuser]# ls -ld demodir/
drwxr-xr-t 2 testuser testuser 4096 demodir/
```

Удаляем установленные разрешения также с помощью команды *chmod*:

```
# chmod u-s file.txt
# chmod g-s file.txt
# chmod -t demodir
```

Проверяем изменения:

```
# ls -l
```

```
[root@alt testuser]# ls -l
итого 4
drwxr-xr-x 2 testuser testuser 4096 demodir
-rwx--x--- 1 testuser testuser 0 file.txt
```

Теперь установим расширенные и базовые права доступа одновременно.

Создадим в домашнем каталоге пользователя *testuser* файл *test.txt* с текстом “Установка SUID” и файл *test2.txt* с текстом “Установка SGID”.

Установим для файла *test.txt* разрешение *SUID* с базовыми правами доступа 755:

```
# chmod 4755 test.txt
```

Установим для файла *test2.txt* разрешение *SGID* с базовыми правами доступа 644:

```
# chmod 2755 test2.txt
```

Проверяем изменения:

```
# ls -l test.txt test2.txt
```

```
[root@alt testuser]# ls -l test.txt test2.txt
-rwxr-sr-x 1 testuser testuser 24 test2.txt
-rwsr-xr-x 1 testuser testuser 24 test.txt
```

5. Пользовательская маска

Когда Вы создаете новый файл или каталог, ему назначаются начальные разрешения. На эти начальные разрешения влияют два фактора:

1. Создаете Вы обычный файл или каталог.
2. Текущая пользовательская маска.

Пользовательская маска — восьмеричная битовая маска, которая используется для очистки разрешений новых файлов и каталогов, создаваемых процессом. Если в маске установлен какой-либо бит, то соответствующее разрешение в новых файлах очищается. Например, маска 0002 очищает бит записи для остальных пользователей. Начальные нули показывают, что специальные разрешения пользователя и группы не удаляются. Маска 0077 очищает все разрешения группы и остальных пользователей для новых файлов.

Для установки маски прав доступа по умолчанию для новых файлов и каталогов используется команда *umask*.

Команда *umask* без аргументов отображает текущее значение пользовательской маски командной оболочки.

Зайдем в учетную запись *testuser* и выполним команду *umask* без аргументов:

```
# su - testuser
```

```
$ umask
```

```
[testuser@alt ~]$ umask  
0022
```

Также создадим папку *demodir2* и файл *file2.txt*:

```
$ mkdir demodir2
```

```
$ touch file2.txt
```

Посмотрим права доступа для каталога *demodir2* и файла *file2.txt*:

```
$ ls -ld demodir2
```

```
$ ls -l file2.txt
```

```
[testuser@alt ~]$ ls -ld demodir2  
drwxr-xr-x 2 testuser testuser 4096 demodir2  
[testuser@alt ~]$ ls -l file2.txt  
-rw-r--r-- 1 testuser testuser 0 file2.txt
```

Для новых каталогов *umask* отнимает 0022, что дает права доступа *rwxr-xr-x*, для новых файлов — *rw-r--r--*, т.к. файлы по умолчанию не получают права на выполнение.

Значение маски по умолчанию для пользователей задается сценариями запуска командной оболочки. По умолчанию для всех пользователей в ОС Альт пользовательская маска устанавливается в файле */etc/profile*.

```
$ cat /etc/profile
```

Пользователи могут переопределять системные значения по умолчанию в файлах *.bash_profile* и *.bashrc* в своих домашних каталогах.

Для изменения маски текущей командной оболочки используется команда *umask* с одним числовым аргументом. Числовой аргумент должен быть восьмеричным значением, соответствующим новой маске. Можно опустить ведущие нули в маске.

Временно зададим маску 0027:

```
$ umask 027
```

Проверим установку маски:

```
$ umask
```

```
[testuser@alt ~]$ umask  
0027
```

Теперь создадим каталог *demodir3* и файл *file3.txt*:

```
$ mkdir demodir3
```

```
$ touch file3.txt
```

Посмотрим права доступа для каталога *demodir3* и файла *file3.txt*:

```
$ ls -ld demodir3
```

```
$ ls -l file3.txt
```

```
[testuser@alt ~]$ ls -ld demodir3
drwxr-x--- 2 testuser testuser 4096 demodir3
[testuser@alt ~]$ ls -l file3.txt
-rw-r----- 1 testuser testuser 0 file3.txt
```

Для новых файлов владелец получает разрешения на чтение и запись, а группа-владелец получает разрешение только на чтение. Остальные пользователи не получают никаких разрешений.

Для новых каталогов владелец получает все разрешения, группа-владелец получает разрешение на чтение и выполнение, а остальные пользователи не получают никаких разрешений.

6. Списки контроля доступа (ACL)

Для реализации сложных структур прав доступа используются расширенные права — *ACL (Access Control List)* — списки контроля доступа), которые дают большую гибкость, чем стандартный набор полномочий.

Поддержка *ACL* на файловых системах *ext4* и *XFS* включена по умолчанию. При необходимости можно отключить поддержку списков доступа, указав для выбранной файловой системы опцию “*noacl*” в файле */etc/fstab*.

ACL предоставляет дополнительные функциональные возможности ОС, НО у нее есть один недостаток — не все утилиты ее поддерживают. Следовательно, можно потерять настройки *ACL* при копировании или перемещении файлов, а ПО для резервного копирования может не выполнить резервное копирование настроек *ACL*.

Для установки и настройки списка доступа используется команда *setfacl*.

Синтаксис команды *setfacl*:

```
$ setfacl [options] acl_spec file | directory
```

где *acl_spec* — спецификация *ACL*, указывающая, какие права доступа применять.

Основные правила команды *setfacl*:

u:username:permissions — установка разрешений для пользователя;

g:groupname:permissions — установка разрешений для группы;

o:permissions — установка разрешений для всех остальных пользователей;

m:permissions — установки маски разрешений.

Основные опции команды *setfacl*:

Опция *-m* изменяет/добавляет правила *ACL*.

Опция *-x* удаляет правило *ACL*.

Опция *-b* удаляет ВСЕ правила *ACL*.

Опция *-R* рекурсивно применяет операции ко всем подкаталогам и файлам.

Опция `-l` позволяет применять изменения без изменения стандартных прав доступа, т.е. игнорировать маску.

Зайдем в учетную запись пользователя *testuser*.

Установим для пользователя *user* право на чтение и на запись файла *file2.txt*:

```
$ setfacl -m u:user:rw file2.txt
```

Чтобы увидеть текущие настройки *ACL*, используется команда *getfacl*.

Посмотрим установленные настройки на файле без *ACL* и сравним вывод с выводом команды *ls -l*:

```
$ getfacl file.txt
```

```
$ ls -l file.txt
```

```
[testuser@alt ~]$ getfacl file.txt
# file: file.txt
# owner: testuser
# group: testuser
user::rw-
group::r--
other::r--

[testuser@alt ~]$ ls -l file.txt
-rw-r--r-- 1 testuser testuser 0 file.txt
```

Права доступа отображаются по-разному, но значения соответствующих полей одинаковы.

Теперь попробуем посмотреть установленные настройки на файле с установленными правилами *ACL* и сравним вывод с командой *ls -l*:

```
$ getfacl file2.txt
```

```
$ ls -l file2.txt
```

```
[testuser@alt ~]$ getfacl file2.txt
# file: file2.txt
# owner: testuser
# group: testuser
user::rw-
user:user:rw-
group::r--
mask::rw-
other::r--

[testuser@alt ~]$ ls -l file2.txt
-rw-rw-r--+ 1 testuser testuser 0 file2.txt
```

Если у файла или каталога установлены *ACL*, то в выводе команды *ls -l* будет отображаться знак “+”.

Команда *ls* не выводит имя второго владельца файла и его права доступа к файлу, хотя второй владелец присутствует, как показывает команда *getfacl*.

Попробуем сделать копию файла *file2.txt*:

```
$ cp file2.txt file2cp.txt
```

Посмотрим права доступа файла *file2cp.txt*:

```
$ ls -l file2cp.txt
```

```
[testuser@alt ~]$ ls -l file2cp.txt
-rw-r--r-- 1 testuser testuser 0 file2cp.txt
```

Настройки *ACL HE* копируются.

При работе с *ACL* нужно быть внимательнее, т.к. не все программы могут использовать *ACL*.

Заметим, что для копирования файла с сохранением *ACL* необходимо использовать опцию *-p* для команды *cp*:

```
$ cp -p file2.txt file2cp2.txt
```

Теперь посмотрим права доступа файла *file2cp2.txt*:

```
$ ls -l file2cp2.txt
```

```
[testuser@alt ~]$ ls -l file2cp2.txt
-rw-rw-r--+ 1 testuser testuser 0 file2cp2.txt
```

Чтобы задать максимальные права доступа для всех пользователей, необходимо использовать маску *ACL*. Например, маска *r--* показывает, что пользователи и группы не могут иметь больших прав, чем просто чтение, даже если им назначены права доступа на чтение и запись.

Установим максимальные права доступа для пользователя *user* на файл *file2.txt*:

```
$ setfacl -m u:user:rwX file2.txt
```

Затем зададим маску *ACL*:

```
$ setfacl -m m:r-- file2.txt
```

Посмотрим права доступа на файл *file2.txt*:

```
$ getfacl file2.txt
```

```
[testuser@alt ~]$ getfacl file2.txt
# file: file2.txt
# owner: testuser
# group: testuser
user::rw-
user:user:rwX          #effective:r--
group::r--
mask::r--
other::r--
```

Права на файл устанавливаются в соответствии с выполняемой командой, но маска, установленная как *r--*, лишает разрешения на запись и выполнение всех владельцев, кроме основного. Это подтверждает появившаяся запись в выводе команды *getfacl*.

Для отмены маски ее следует установить в значение *rwX*.

Полностью удалим все *ACL* правила из файла *file2.txt*:

```
$ setfacl -b file2.txt
```

Проверим права доступа файла *file2.txt*:

```
$ ls -l file2.txt
```

```
[testuser@alt ~]$ ls -l file2.txt
-rw-r--r-- 1 testuser testuser 0 file2.txt
```

7. Базовые атрибуты файлов

Атрибуты файлов — метаданные, которые определяют различные свойства и поведение файлов и каталогов в файловой системе. Другими словами, атрибуты управляют тем, как файловая система обращается с файлами и каталогами, т.е. с объектами файловой системы. Например, атрибуты могут помечать файлы как неизменяемые или доступные только для добавления.

Атрибуты делятся на базовые и расширенные.

Базовые атрибуты поддерживаются файловыми системами *ext2*, *ext3*, *ext4*, *xf*s и *btrfs*, но не поддерживаются на *FAT32*, *NTFS* и других файловых системах, не адаптированных под *POSIX*.

Для отображения установленных атрибутов файлов и каталогов используется команда *lsattr*.

Синтаксис команды *lsattr*:

```
$ lsattr [options] file | directory
```

Чтобы отобразить атрибуты файлов в текущем каталоге, выполним команду *lsattr* без аргументов:

```
$ lsattr
```

Чаще всего команда *lsattr* принимает имена файлов и каталогов для проверки в качестве аргументов.

Отобразим атрибуты файла *file.txt*:

```
$ lsattr file.txt
```

```
[testuser@alt ~]$ lsattr file.txt
-----e----- file.txt
```

В общем, выходные данные команды *lsattr* состоят из 2-х столбцов: атрибуты, имя файла или каталога. В столбце *attributes* показан ряд букв, представляющих различные параметры и свойства, которые установлены или отключены для каждого файла или каталога.

Атрибут *e* указывает на то, что файл *file.txt* использует непрерывный диапазон блоков для сопоставления блоков хранения.

Основные опции команды *lsattr*:

1. Опция *-R* рекурсивно перечисляет атрибуты всех файлов в каталоге.
2. Опция *-a* перечисляет только файлов с установленными атрибутами.

Выполним команду *lsattr* с опцией *-a*:

```
$ lsattr -a
```

3. Опция *-d* выводит список только каталогов, а не их содержимого.

Выполним команду *lsattr* с опцией *-d*:

```
$ lsattr -d
```

```
[testuser@alt ~]$ lsattr -d
-----e----- .
```


4. Опция `-l` позволяет вывести полные имена атрибутов вместо односимвольных сокращений.

```
$ lsattr -l
```

Важно помнить, что при копировании файлов с использованием команд `cp`, `rsync` (без дополнительных опций) базовые атрибуты не сохраняются.

Команда `chattr` позволяет изменять базовые атрибуты файлов, поддерживаемые файловыми системами `ext3`, `ext4`.

Синтаксис команды `chattr`:

```
$ chattr [options] WhatWhich file | directory
```

где What: + добавление, - удаление, = точная установка;

Which: атрибуты.

Основные опции команды `chattr`:

Опция `-R` позволяет рекурсивно изменять атрибуты каталогов и их содержимого (символические ссылки игнорируются).

Опция `-V` позволяет вывести дополнительную информацию.

Основные базовые атрибуты:

Символ	Название	Описание
a	Append Only	Только добавление данных в конец файла
A	No Atime Updates	Не обновлять время последнего доступа к файлу
c	Compressed	Автоматическое сжатие файла
C	Copy-on-write Off	Отключить механизм "копирование при записи"
d	No Dump	Исключение из резервных копий (dump)
D	Sync Directories	Синхронное обновление каталогов
i	Immutable	Неизменяемый файл (нельзя удалить, переименовать, изменить)
j	Data Journaling	Журналирование данных перед записью в файл
s	Secure Deletion	Содержимое файла полностью затирается при удалении

S	Synchronous Updates	Все операции записи выполняются синхронно
u	Undeleteable	Файл не удаляется полностью, что позволяет его восстановить

Некоторые атрибуты требуют выполнение команды *chattr* от имени суперпользователя.

Зайдем в учетную запись пользователя *testuser*.

Установим атрибут *i* для файла *file.txt*:

```
$ chattr +i file.txt
```

```
[testuser@alt ~]$ chattr +i file.txt
chattr: Операция не позволена while setting flags on file.txt
```

Команда *chattr* возвращается с ошибкой.

Зайдем под учетной записью суперпользователя и добавим атрибут *i* для файла *file.txt*:

```
# cd /home/testuser
```

```
# chattr +i file.txt
```

Проверим изменения:

```
# lsattr file.txt
```

```
[root@alt testuser]# lsattr file.txt
----i-----e----- file.txt
```

Теперь попробуем удалить файл *file.txt*:

```
# rm file.txt
```

```
[root@alt testuser]# rm file.txt
rm: удалить пустой обычный файл 'file.txt'? y
rm: невозможно удалить 'file.txt': Операция не позволена
```

Когда файл *file.txt* имеет атрибут *i*, то даже после согласия на удаление, команда *rm* не сможет выполнить эту операцию.

Снимем все атрибуты с файла *file.txt*:

```
# chattr = file.txt
```

Проверим, что атрибуты сняты:

```
# lsattr file.txt
```

```
[root@alt testuser]# lsattr file.txt
----- file.txt
```

Создадим журнал логов *journal.txt*:

```
# touch journal.txt
```

Установим атрибут *a* для файла *journal.txt*, чтобы можно было только добавлять данные в файл, но не изменять или удалять существующие данные:

```
# chattr +a journal.txt
```

Проверим изменения:

```
# lsattr journal.txt
```

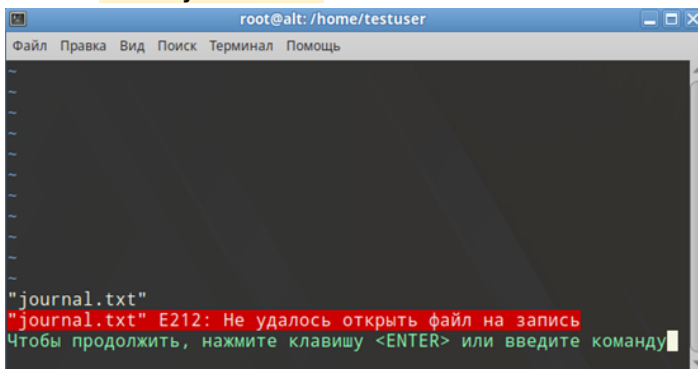
```
[root@alt testuser]# lsattr journal.txt
-----a-----e----- journal.txt
```

Добавим в файл строку «Проверка работы атрибута +a»:

```
# echo "Проверка работы атрибута +a" >> journal.txt
```

Отредактируем файл *journal.txt*:

```
# vim journal.txt
```



Редактирование файла не будет возможно. Пользователь может только добавить в него данные, но изменить уже существующие записи нельзя.

Удалить файл *journal.txt* также будет невозможно:

```
# rm journal.txt
```

```
[root@alt testuser]# rm journal.txt
rm: удалить обычный файл 'journal.txt'? y
rm: невозможно удалить 'journal.txt': Операция не позволена
```

Установим атрибут *d* для каталога */tmp*. Каталог */tmp* содержит временные файлы, которые не должны попадать в резервные копии для экономии места.

```
# chattr +d /tmp
```

Проверим изменения:

```
# lsattr -d /tmp
```

```
[root@alt testuser]# lsattr -d /tmp
-----d----- /tmp
```

8. Расширенные атрибуты файлов

Расширенные атрибуты предоставляют больше информации о файле, чем базовые атрибуты.

Расширенные атрибуты хранятся в метаданных файлов в формате ключ:значение. Каждому атрибуту присваивается уникальный ключ (имя атрибута), а значение может быть любым (строка, число, бинарные данные и т.д.).

Пример:

Атрибут *user.comment* имеет значение "*Top secret*", где *user.comment* — ключ, а "*Top secret*" — значение.

Важно помнить, что стандартные команды копирования *cp* и *rsync* не сохраняют расширенные атрибуты по умолчанию! Чтобы сохранить расширенные атрибуты при копировании, нужно использовать специальные опции.

Специальные опции для сохранения расширенных атрибутов:

Символ	Параметры для сохранения атрибутов
cp	--preserve=xattr
rsync	--xattrs
tar	--xattrs и --xattrs-include=*

Расширенные атрибуты делятся на несколько классов, каждый из которых имеет свое назначение.

Класс	Применение
security	Задание привилегий для процессов (например, capabilities).
system	Системные атрибуты, недоступные обычным пользователям.
trusted	Атрибуты, используемые привилегированными процессами (например, SELinux).
user	Пользовательские атрибуты, доступные для чтения и записи без специальных прав.

Для работы с расширенными атрибутами файлов установим пакет *attr*:

```
# apt-get update && apt-get install attr
```

Зайдем под учетной записью пользователя *testuser*.

Для просмотра расширенных атрибутов файла используется команда *getfattr*.

Сначала создадим пустой файл *example.txt*:

```
$ touch example.txt
```

Отобразим все расширенные атрибуты файла *example.txt*:

```
$ getfattr -d example.txt
```

Если атрибутов нет, то вывод будет пустым.

```
[testuser@alt ~]$ getfattr -d example.txt
[testuser@alt ~]$
```

Установим атрибут *user.comment* со значением "Top secret" для файла *example.txt*:

```
$ setfattr -n user.comment -v "Top secret" example.txt
```

Опция *-n* указывает на ключ, а опция *-v* — значение.

Снова отобразим атрибуты файла *example.txt*:

```
$ getfattr -d example.txt
```

```
[testuser@alt ~]$ getfattr -d example.txt
# file: example.txt
user.comment="Top secret"
```

Удалим атрибут *user.comment* из файла *example.txt*:

```
$ setfattr -x user.comment example.txt
```

Проверим, что атрибут был удален:

```
$ getfattr -d example.txt
```

```
[testuser@alt ~]$ getfattr -d example.txt
[testuser@alt ~]$
```

Расширенные атрибуты можно использовать для хранения дополнительной информации.

Используем расширенный атрибут для контроля целостности файла.

Вычислим хэш-сумму файла *example.txt*:

```
$ md5sum example.txt
```

Хэш-сумма пустого файла *d41d8cd98f00b204e9800998ecf8427e*.

```
[testuser@alt ~]$ md5sum example.txt
d41d8cd98f00b204e9800998ecf8427e  example.txt
```

Добавим хэш-сумму как расширенный атрибут для файла *example.txt*:

```
$ setfattr -n user.checksum -v $(md5sum example.txt | cut -d' ' -f1) example.txt
```

Проверим, что хэш-сумма успешно добавлена как атрибут файла *example.txt*:

```
$ getfattr -n user.checksum example.txt
```

```
[testuser@alt ~]$ getfattr -n user.checksum example.txt
# file: example.txt
user.checksum="d41d8cd98f00b204e9800998ecf8427e"
```

Убедимся, что при копировании файлов с использованием специальных опций атрибуты сохраняются.

Скопируем файл *example.txt* в тот же каталог с именем *example2.txt*:

```
$ rsync -a --xattrs example.txt example2.txt
```

Опция *-a* сохраняет владельцев, права, временные метки и символьные ссылки.

Опция *--xattr* сохраняет расширенные атрибуты файлов.

Проверим, что атрибуты сохранились в файле *example2.txt*:

```
$ getfattr -d example2.txt
```

```
[testuser@alt ~]$ getfattr -d example2.txt
# file: example2.txt
user.checksum="d41d8cd98f00b204e9800998ecf8427e"
```